

Archiving Years of Dataverse Audit History Before You Prune It — A Pragmatic, Open-Source Pattern

A note up-front. This post documents an architectural choice — not a Microsoft product, not a Microsoft offering, and not officially supported. For most Dataverse-to-Azure replication needs the first-class options are [Azure Synapse Link for Dataverse](#) and [Microsoft Fabric Link for Dataverse](#); if either fits your scenario, prefer them.

The specific gap this post addresses is the **audit table**. Synapse Link / Fabric Link can replicate audit *header* rows like any other table, but the per-change payload returned by `RetrieveAuditDetails` (old value vs new value, attribute mask, related-record diffs) is computed by Dataverse on demand and is not part of the table contents the link replicates. So if your need is "keep the row-level *what changed* evidence after I prune from Dataverse", you need to call `RetrieveAuditDetails` while the source row still exists — which is what this pattern does.

The audience this is written for: enterprises with years of accumulated audit history that, for whatever reason, *couldn't* adopt Synapse Link / Fabric Link — typically because their analytics platform of record sits outside Azure (most commonly Snowflake), or because internal review hasn't approved the link in their tenant — and who need a defensible cold archive *before* Dataverse's audit-deletion job runs. Treat this as a reference implementation you can adapt, fork, or discard.

Why the audit table is special

Most of what Dataverse exports through Synapse Link is *current state*. The **audit table** is different in two ways:

1. It's an immutable, append-only record of *who changed what, when, and from where* — the closest thing Dataverse has to a forensic ledger.
2. The valuable part of an audit row is not the row itself; it's the diff (old value → new value, attribute mask, related-record context) returned by the bound `RetrieveAuditDetails` function. That diff is computed at read-time against the still-living source row. Once Dataverse deletes the audit, no API call can reconstruct it.

That combination makes the audit table the single most useful Dataverse table for:

- Regulatory and compliance investigations
- "Why did this opportunity status change in Q3 of 2022?" forensic queries (years after the fact)
- Internal analytics on user behaviour and process adoption

It's also the table that grows the fastest. The Dataverse retention default is 90 days, but in practice many enterprises change that — they set it to several years, or to *never delete* — because nobody wants to be the admin who threw away evidence. The result, often after a quiet five or seven years, is an audit table sitting on tens of GB to multiple TB of capacity, dominating the entitlement bill, and never touched in normal operations.

At that point the storage conversation becomes unavoidable. The realistic choices are:

- 1. **Keep buying entitlement.** Predictable, but unbounded.
- 2. **Move the cold tail somewhere cheaper that you control, then let Dataverse's audit-deletion job reclaim the space.** The hot months stay in Dataverse where users expect them; the years of historical evidence live in your own storage account, queryable when you need them.

This pattern is for option 2 — specifically, for the *one-time bulk export of multi-year history*, with the option to keep a slow trickle running afterwards if you want to top up.

A crucial point that often gets lost: this pipeline does not need to run live. It is perfectly reasonable to be deliberately months or years behind real-time. The goal is to get a defensible copy of *cold* data out — the rows you are about to allow Dataverse to delete — not to mirror the audit feed in real time.

What "good" looks like for an external audit copy

Before showing any code, here's the rubric I held this design to. If you build your own, hold yours to the same rubric:

Property	Why it matters
Idempotent	Re-running the same time window must not duplicate rows. Network blips happen.
Crash-safe (exactly-once-effective)	If the process dies mid-window, the next run must replay the same window cleanly. The watermark advances <i>only</i> after the data is durable in the destination.
Bounded memory	A backlog of millions of audits cannot be loaded all at once.
Backpressure-aware	Dataverse rate-limits aggressively. Throttle responses must not drop rows.
Observable	Every window logs <code>[entity] mode=BACKLOG/LIVE, lag=Nmin, window=10min, records=N</code> so you can watch it work.
Sink-agnostic	The "where does it land" decision is config, not code. Storage choices change; the orchestrator shouldn't.
Field-level discretion	Audit details can carry PII. The pattern should let admins narrow which attributes leave the platform.

The pattern

The pipeline is conceptually four stages, repeated per entity, per time window:

```
Per entity (account, contact, ...):  
  1. Read watermark (lastSyncEnd) from sink's state container  
  2. Compute next time window: [lastSyncEnd, lastSyncEnd + N min)  
  3. Fetch audit headers from Dataverse Web API:  
     $filter=createdon ge <start> and createdon lt <end>  
  4. For each header → call RetrieveAuditDetails (bound function)  
  5. (Optional) Project to allow-listed attributes only  
  6. Upsert each audit doc into sink (id = auditid, idempotent)  
  7. Advance watermark ONLY if step 6 fully succeeded  
  
Loop until caught up; then either exit (function) or sleep (daemon)
```

Three details in this picture do most of the resiliency work. They are deceptively simple:

Detail 1: Half-open time windows (ge / lt)

```
[09:00:00, 09:10:00) → window 1  
[09:10:00, 09:20:00) → window 2
```

The boundary moment (09:10:00.000) belongs to window 2, not window 1. So adjacent windows never overlap and never gap, no matter how many times you replay. This is the same trick Kafka uses for offsets — it's why you can run the loop with confidence.

Detail 2: The destination document key is the Dataverse auditid GUID

Dataverse already assigns a globally unique GUID to every audit row. That GUID becomes the document `id` in the sink. So when you upsert the same audit twice, the second write is a no-op overwrite of the first — idempotency for free, no client-side dedupe table to maintain.

Detail 3: The watermark moves *after* the write, not with it

The naive version of this pipeline does:

```
records = fetch_window(...)  
sink.write(records)           # 950/1000 succeed; 50 fail with 429  
sink.update_watermark(window_end) # ← BUG: window now "done", 50 rows lost forever
```

The resilient version raises a typed exception when *any* record fails, and the watermark update is conditional on a clean write:

```

try:
    written = sink.write(records)
    sink.update_watermark(window_end)
except SinkPartialWriteError:
    # Watermark stays put. Next loop iteration replays the same window.
    # Rows that already landed become no-op upserts. Rows that failed get retried.
    raise

```

This single change is the difference between "best effort" and "exactly-once-effective." It's also the mistake I've seen most often when people roll their own.

Architecture in a little more detail

The four-stage diagram above hides a few decisions that matter in practice. This section walks through them so you can decide whether they fit your environment before you fork.

The exact API call shape

The pipeline talks to Dataverse over the standard Web API. There are exactly two call types per entity per window:

1. Audit header query — one paged GET per window:

```

GET https://{org}.crm.dynamics.com/api/data/v9.2/audits
    ?$filter=objecttypecode eq 'account'
      and createdon ge 2026-01-01T00:00:00Z
      and createdon lt 2026-01-01T00:10:00Z
      &$select=auditid,createdon,operation,action,_userid_value,objecttypecode
      &$orderby=createdon asc
Prefer: odata.maxpagesize=1000
Authorization: Bearer <token>

```

The combination of `ge / lt` (half-open) plus `$orderby=createdon asc` plus `odata.maxpagesize` gives you deterministic, resumable paging. If a page fails mid-stream, the next attempt at the same window starts from the same first row.

2. Audit detail enrichment — one bound-function call per audit row:

```

GET https://{org}.crm.dynamics.com/api/data/v9.2/
    audits({auditid})/Microsoft.Dynamics.CRM.RetrieveAuditDetails()
Authorization: Bearer <token>

```

This is where the *forensic* content lives — `OldValue`, `NewValue`, `ChangedAttributes`, related-record context. It's also the most expensive call in the pipeline (one per row, computed at read-time by Dataverse). Concurrency is bounded with an `asyncio.Semaphore` per entity to keep parallel requests

below whatever your environment's service-protection limit allows. The reference default is conservative — raise it only after you've watched a clean run.

Why concurrency lives at the entity level, not the row level

Dataverse's [service-protection limits](#) are evaluated **per user, per Web API endpoint, per five-minute window**. Hammering one entity with hundreds of parallel `RetrieveAuditDetails` calls is the fastest way to get throttled (HTTP 429 with a `Retry-After` header). Spreading the same total concurrency budget across multiple entities — say, 4 entities × 8 in-flight detail calls each — keeps every individual entity below its protection threshold while still using the network. That's why the reference orchestrator runs entities concurrently with `asyncio.gather` and uses a per-entity semaphore inside, instead of one giant pool.

When you do get a 429, the SDK / `aiohttp` retry path honours the `Retry-After` value rather than backing off blindly. This matters: a fixed 30-second back-off when Dataverse is asking for 2 seconds is a 15× throughput tax for no reason.

State container schema

The state container is intentionally tiny. One document per entity, partition key = entity name, payload of:

```
{
  "id":          "account",
  "entity":      "account",
  "lastSyncEnd": "2026-01-15T14:30:00",
  "recordCount": 2438109,
  "updatedAt":   "2026-01-15T14:30:02.114Z"
}
```

The orchestrator reads it once at the top of every entity loop and writes it once at the bottom — *only if* the data write succeeded. The write is a `replace_item` on a known `id`, which makes the operation atomic from the sink's point of view: either the new watermark is durable, or the old one is. There is no "half-updated" state to recover from.

For sinks that don't have a native key/value store (ADLS Gen2, OneLake, Snowflake), the state document lands in a parallel location — `_state/<entity>.json` for blob/lake, or a `audit_sync_state` table for Snowflake. The contract from the orchestrator's side is identical: `get_state(entity) → datetime | None`, `update_state(entity, end, count) → None`.

Why `auditid` makes a perfect document key

The `auditid` GUID assigned by Dataverse has three properties that matter for an idempotent sink:

1. **Unique across the entire tenant.** No collisions across entities, environments, or replays.

2. **Assigned at write-time by the source.** Not generated client-side, so identical for every consumer of the same audit row.
3. **Stable.** Dataverse never reissues a GUID for the same logical event.

Using it as the destination document `id` collapses three classes of bug into a no-op:

- *Replayed window after a crash:* upsert overwrites itself.
- *Network blip causing a duplicate write inside the same loop:* upsert overwrites itself.
- *Two parallel orchestrators racing the same backlog:* both produce the same final state.

The alternative — generating a client-side hash, or counting on a separate dedupe table — introduces operational surface area you do not need.

How the half-open math actually works

The orchestrator stores `lastSyncEnd` as the *exclusive upper bound* of the most recent successful window. Every new window is therefore:

```
start = lastSyncEnd           (inclusive)
end   = lastSyncEnd + windowSizeMinutes (exclusive)
filter: createdon ge start AND createdon lt end
```

The value at `lastSyncEnd` itself is *never* re-fetched, because it was already covered by the previous window's exclusive `lt`. The value at `end` is *never* skipped, because the next window's inclusive `ge` will cover it. This holds true no matter how many times you replay, how many parallel workers run on different entities, or what timezone the server is reporting. (The reference implementation pins everything to UTC ISO-8601, with no timezone conversions, to remove one entire category of foot-gun.)

When the watermark refuses to advance

The `SinkPartialWriteError` raised by `cosmos_sink.write_audits` (and the equivalent contracts in the other sinks) carries:

- `entity` — which entity failed
- `written` — how many rows in this batch did succeed
- `failed` — how many failed
- `sample_errors` — up to three `(audit_id, status_code, message)` tuples for the log

The orchestrator catches it, logs at `ERROR`, and re-raises. The outer `process_entity_continuous` loop's `except` block then declines to assign `last_sync_end = window_end`. Result: the same window will be replayed on the next iteration, and the rows that already landed become no-op upserts on their existing `id`. Nothing is lost. Nothing is duplicated.

This is the single most important behavioural property of the pipeline. It is also a one-line decision — *raise vs return a count* — and in the wild I've seen it implemented the wrong way more often than the right way.

A worked test run

The reference implementation ships with a small harness for validating the loop end-to-end before you point it at production. The shape of a clean run looks like this.

1. Smoke test: one entity, two-hour window

```
# unified-deployment/smoke_test_cosmos.py
# Configures a single entity (systemuser) and a 2-hour window.
# Expected first-run output: a few records written. Expected re-run: zero.
python smoke_test_cosmos.py
```

First-run log (abridged):

```
[systemuser] Mode=BACKLOG, lag=120m, window=120min
[systemuser] Fetched 3 audits for window 2026-01-15T12:00:00Z .. 2026-01-15T14:00:00Z
[systemuser] cosmos sink: wrote 3 records
[systemuser] State updated: lastSyncEnd=2026-01-15T14:00:00
```

Re-running the same script immediately:

```
[systemuser] Mode=BACKLOG, lag=0m, window=10min
[systemuser] Fetched 0 audits for window 2026-01-15T14:00:00Z .. 2026-01-15T14:10:00Z
[systemuser] cosmos sink: wrote 0 records
```

Proof of two things at once: the window math is correct (the second run's start = the first run's end), and the sink is genuinely idempotent (nothing was rewritten because no new rows existed).

2. Partial-failure drill: prove the watermark holds

The `SinkPartialWriteError` contract is the part of the pipeline most worth proving in a controlled setting. The reference repo includes `test_partial_failure.py`, a three-phase test that does the following with no live Dataverse traffic:

Phase	What it does	What it asserts
1. Fault injection	A <code>FakeContainer</code> is wired in; the 3rd of 5 <code>upsert_item</code> calls raises a synthetic <code>CosmosHttpResponseError(503)</code> .	All 5 records are <i>attempted</i> (the loop does not bail on the first failure), and the sink raises <code>SinkPartialWriteError(written=4, failed=1)</code> after the loop.
2. Recovery	A <code>HealthyContainer</code> is wired in for the retry.	All 5 records land cleanly with their original <code>auditid</code> -based ids — no duplicates, no orphans.
3. End-to-end watermark hold	A <code>FailingSink</code> that always raises <code>SinkPartialWriteError</code> is plugged into a real <code>process_window</code> call (with monkey-patched <code>fetch_audits</code> and <code>fetch_audit_details_with_retry</code>).	The exception propagates out of <code>process_window</code> , and <code>sink.get_state("account")</code> is still <code>None</code> afterwards — i.e., the watermark was <i>not</i> advanced.

A passing run on this test is the strongest local evidence that the "refuse to advance state on partial failure" contract holds end-to-end. If you fork this code and change the sink, run this test against your fork before you trust it with anything real.

3. Realistic backlog: drain a multi-window range

For a sandbox check that exercises the full backlog→catch-up behaviour, the harness also includes `full_backlog_cosmos.py`. It pins `windowSizeMinutes.backlog = 10` and `modeAutoDetect.enabled = false`, runs across all configured entities, and prints a per-entity breakdown afterwards:

```
[systemuser] Mode=BACKLOG, lag=...m, window=10min, records=...
[contact]    Mode=BACKLOG, lag=...m, window=10min, records=...
[account]    Mode=BACKLOG, lag=...m, window=10min, records=...
...

=====
Total records  <total>
Elapsed       <wall time>
Throughput    <records / sec>
Per-entity:
  account     <count>   lastSyncEnd=<iso timestamp>
  contact     <count>   lastSyncEnd=<iso timestamp>
  systemuser  <count>   lastSyncEnd=<iso timestamp>
=====
```

When we exercised this on a sandbox tenant covering the most recent 90 days, the run completed in single-digit minutes at a few hundred records per second — with the operational signal being not the absolute throughput but the *steady cadence of per-window log lines*. If those go ragged, look at lag, sink errors, or 429s before you scale up `max_concurrent_entities`.

4. What to look for in a successful run

- **Per-window log lines tick predictably.** If the gap between adjacent `[entity] Fetched N audits` lines suddenly grows, you're either being throttled or the sink is back-pressuring. Both are visible in the same log stream.
- **`lastSyncEnd` advances monotonically.** Pull the state container after the run; if any entity's `lastSyncEnd` went backwards, something is very wrong (almost always an operator running the wrong config against the wrong destination).
- **A second run of the same range writes zero rows.** This is the idempotency check, and it should be the first thing you do after a backlog completes.
- **Sink-side document counts match `recordCount` in state.** If they diverge, a partial-write was swallowed somewhere — chase down which sink, which entity, and whether your `SinkPartialWriteError` contract is honoured.

How to adapt it

The whole point of publishing the pattern is so people change it for their own environment. Here are the most common adaptations and roughly what they cost.

Add or remove an entity

Edit `unified-deployment/config.json`:

```
{
  "entities": [
    { "name": "account",      "attributes": ["name", "telephone1"] },
    { "name": "contact",     "attributes": ["fullname", "emailaddress1"] },
    { "name": "yourcustom",  "attributes": [] }    // empty = store everything
  ]
}
```

That's it — the orchestrator picks the new list up on next start, and the sink will create per-entity state on first write.

Narrow which fields leave Dataverse

The `attributes` array on each entity is an allow-list applied *inside* `_filter_audit_detail` before any sink ever sees the row. The contract:

- Empty list (or missing) → store the full `RetrieveAuditDetails` payload as-is.
- Populated list → prune `OldValue`, `NewValue`, and `ChangedAttributes` to the named attributes only. System keys starting with `@` (e.g., `@odata.type`) are always preserved.
- Non-attribute audit detail types (Relationship, Action) pass through untouched.

This is the right place to enforce data-residency or PII rules — the regulated fields never reach the sink, never reach the destination subscription, never reach a Power BI report two hops downstream.

Add a new sink (e.g., your own data lake or warehouse)

A new sink is one new file plus one factory entry. The contract you implement is the `AuditSink` ABC in `unified-deployment/sinks/base.py`:

```
from .base import AuditSink, SinkPartialWriteError

class MyCustomSink(AuditSink):
    name = "mycustom"

    def initialize(self) → None:
        # create database / table / filesystem if missing
        ...

    def get_state(self, entity: str) → Optional[datetime]:
        # return last successful window_end, or None if never run
        ...

    def update_state(self, entity, last_sync_end, record_count) → None:
        # persist new watermark + cumulative count, atomically
        ...

    def write_audits(self, entity, records, window_end, run_id) → int:
        failures = []
        written = 0
        for r in records:
            try:
                self._upsert_idempotent(audit_id=r["auditid"], doc=r)
                written += 1
            except Exception as e:
                failures.append((r["auditid"], -1, str(e)))
        if failures:
            raise SinkPartialWriteError(entity, written, len(failures), failures)
        return written
```

Register it in `unified-deployment/sinks/__init__.py`:

```
def get_sink(config):
    sink_type = config["sink"]["type"]
    if sink_type == "mycustom":
        from .mycustom_sink import MyCustomSink
        return MyCustomSink(config)
    ...
```

Flip `config.json`:

```
{ "sink": { "type": "mycustom" } }
```

The two non-negotiable invariants when you write a new sink:

1. **Idempotent on auditid.** Whatever `_upsert_idempotent` does for your destination (`MERGE INTO ... ON audit_id` for a warehouse, `PUT overwrite` for blob storage, `ON CONFLICT DO UPDATE` for Postgres) must leave the destination identical when called twice with the same record.
2. **`SinkPartialWriteError` on any per-row failure.** The orchestrator depends on this exception to refuse the watermark advance. If you swallow exceptions and return a partial count instead, you have re-introduced the exact bug this whole pattern is designed to prevent.

Change the cadence

Two independent knobs in `config.json`:

```
{
  "windowSizeMinutes": { "backlog": 60, "continuous": 10 },
  "modeAutoDetect":    { "enabled": true, "backlog_threshold_minutes": 60 }
}
```

When `modeAutoDetect.enabled` is `true`, the orchestrator picks the backlog window if it's more than `backlog_threshold_minutes` behind real-time, and the continuous window otherwise. Disable auto-detect and the `BACKLOG_MODE` env var controls it explicitly — useful when you're running a deliberate one-off backlog and don't want it to silently switch to small windows after the first hour.

For a multi-year archive job, set both window sizes to whatever balances Dataverse rate-limits against per-window overhead in your tenant. 10 to 60 minutes is the practical band; smaller windows mean more state writes per unit time, larger windows mean a bigger replay cost when one fails.

Run it somewhere

The reference container is intentionally hosting-agnostic. The same image works as:

- **An Azure Function** (Timer-triggered, every N minutes) — best fit for a continuous slow-trickle. Cold-starts are fine for this workload because every window is a fresh database connection anyway.
- **An Azure Container App job** — best fit for one-shot multi-year backlog runs, where you want a generous timeout, full CPU, and the freedom to run a single entity at a time via the `ENTITY` env var on a fan-out of jobs.
- **A scheduled Container Instance / cron / Kubernetes CronJob** — best fit for the "once a quarter, top up the archive" cadence many enterprises actually want.
- **Locally for a one-off backfill.** Run it from a workstation if that matches your governance posture; the only persistent state is in the destination sink.

The fan-out trick worth knowing: setting the `ENTITY=account` env var causes the orchestrator to process *only* that entity. Spinning up N copies of the same image with N different `ENTITY` values gives you trivially parallel per-entity scaling without any orchestration changes, and each copy maintains its own watermark independently.

Hook it into observability

The orchestrator emits structured logs by default. Two integrations that pay for themselves:

- **Application Insights** — wire the standard OpenCensus / OpenTelemetry exporter into the logging config. Every `[entity] Fetched N audits` line becomes a queryable trace; partial-write errors become an alert.
- **A simple dashboard query against the state container** — `SELECT entity, lastSyncEnd, recordCount FROM sync_state` (or the equivalent for your sink). When `lastSyncEnd` for any entity stops advancing, you have an incident; when `recordCount` flatlines, you have either a quiet tenant or a silent failure — cross-check with the source.

Neither requires application changes; both should exist before you let the pipeline run unattended.

Choosing where it lands

The orchestrator is sink-agnostic — it talks to a single `AuditSink` interface (`get_state`, `update_state`, `write_audits`) and the destination is a config switch, not a code change. The reference implementation ships with four production-shaped sinks plus a no-op for testing. None of them is *the* answer; they map to platforms enterprises already operate:

Sink	When to consider it
Azure Cosmos DB (NoSQL API)	Operational lookups — "show me everything user X did to record Y in 2022" in milliseconds. Hierarchical partition keys (<code>/entity</code> + <code>/auditYearMonth</code>) keep partitions small as the archive grows over years. Document TTL doubles as a retention policy if you want one. Serverless mode suits a slow-trickle archive workload.
Azure Data Lake Storage Gen2 (Parquet)	The cheap-cold-storage option. Years of audit history land as partitioned Parquet files (<code>entity=.../year=.../month=.../</code>), readable from Fabric notebooks, Synapse Serverless SQL, Databricks, or any Parquet engine. Costs scale with bytes, not throughput — ideal when the archive is rarely queried but must exist.
OneLake (Parquet)	Same Parquet shape as ADLS, but landed inside a Microsoft Fabric Lakehouse. Immediately queryable from a Fabric SQL endpoint, notebooks, and Power BI without further plumbing. The natural choice if your downstream BI is Fabric.
Snowflake (MERGE INTO)	The natural choice when Snowflake is already the analytics platform of record and adding a separate Microsoft analytics estate just for audit data isn't on the table. <code>MERGE INTO ... ON audit_id</code> keeps the same idempotency contract as the Cosmos upsert, and the warehouse stays paused between archival batches.
No-op (logs only)	First-day connectivity testing. Confirms the Dataverse side works before you provision any storage.

A reasonable default split many enterprises arrive at:

- **ADLS Gen2 / OneLake** (or **Snowflake**, if that's your platform) holds the durable historical archive — cheap, partitioned, queryable when (rarely) needed.
- **Cosmos DB** holds the most recent N months for fast operational lookup if there is a use case for it; otherwise skip it entirely.

Adding a sink for storage you already own (e.g., BigQuery, Redshift, on-prem object storage) is roughly 100 lines of Python and one factory entry.

What real-world numbers will look like

The worked test run above shows what a clean sandbox run looks like. Real numbers in your tenant will vary by orders of magnitude depending on:

- How many entities have auditing enabled
- The shape of `RetrieveAuditDetails` calls (more changed attributes per row = more bytes per call)
- Dataverse Web API rate limits applicable to your environment
- Concurrency you allow (`max_concurrent_entities` in the config)
- Sink throughput (Cosmos serverless RU autoscale, ADLS upload bandwidth, Snowflake warehouse size)

The useful operational signal is not the absolute throughput — it's that the throughput is *stable* and the per-window log lines tick predictably. If they don't, look at lag, sink errors, or 429 responses from Dataverse before scaling up concurrency.

Things to be honest about

Because this is a "build it yourself" pattern, here are the trade-offs you take on. These are not flaws — they are the price of doing the export yourself instead of standing up a managed pipeline, and you should price them in:

1. **You own the watermark.** If the destination is wiped without the state container, the next run will start over from whatever seed timestamp you give it (usually the earliest `createdOn` in the audit table for first runs, or your last known `lastSyncEnd` for resumes). Treat the state container as production data — back it up.
 2. **Schema is your problem.** When Dataverse adds a new field to the `audit` table, the change applies in production. Your sink will keep working — the field just shows up in the JSON blob — but if you're projecting columns (e.g., the attribute allow-list), update the config.
 3. **Permissions are your problem.** The application user needs read on `audit` (and `RetrieveAuditDetails` privileges per entity). The Power Platform admin centre handles this, but it's an out-of-band step.
 4. **Cost shape changes.** You're trading "Dataverse storage entitlement" for "ADLS bytes," "Cosmos RUs," or "Snowflake credits." Run the math on your record volume before committing — partitioned Parquet on cool/cold object storage is usually the cheapest by an order of magnitude when the archive is read rarely.
 5. **Auditing your auditor.** If this pipeline becomes evidence in a compliance investigation, the *pipeline itself* needs an audit trail. The reference implementation stamps every output document with a `runId` (UUID per window) and a `processedAt` timestamp. Keep the orchestrator logs.
 6. **It is not a Microsoft product.** I'll repeat this because it matters: if you adopt it, you own the operations, the upgrades, and the on-call pager.
-

When to reach for it (and when not to)

Reasonable fit:

- You have multiple years of accumulated audit history in Dataverse and need to move the cold tail off the platform before pruning to reclaim entitlement.
- The evidence you actually care about is the `RetrieveAuditDetails` payload (old value → new value), not just the audit header row — so a table-level replicator alone wouldn't preserve what you need.

- You're comfortable running this as a *batch job* — once for the historical backfill, then perhaps quarterly or annually to top up — rather than as a live continuous feed. Being deliberately months or years behind real-time is fine and often desirable.
- You want a portable, scriptable export that targets storage you already own — your Azure subscription (ADLS Gen2, OneLake, Cosmos DB), your Snowflake account, or another platform you can write a thin sink for.
- You want field-level control over which attributes leave the platform — useful when audit details contain regulated data.

Not a good fit:

- You only need *current state* of business tables (account, contact, opportunity). Use Synapse Link / Fabric Link — they do exactly that and you don't need this pattern.
- You only need audit *header* rows (who/when/which table) and not the per-attribute change detail. Synapse Link / Fabric Link can replicate the audit table itself; consider that first.
- You need sub-second freshness in the destination. The pattern's natural cadence is one window length (10 min in the reference config); for true real-time, use Dataverse webhooks or change-tracking APIs.
- You don't have somewhere to operate a small Python container, function, or scheduled job — even an annual one.
- You don't have an internal owner who can be paged when the schedule fails.

The reference implementation

The code that backs this post lives at <https://github.com/SweetsNSavories/DataverseAuditLogSyn> under MIT, with no warranty. The `unified-deployment` folder is the version this post describes — single Python codebase, swap sinks via `config.json`, runs locally / in a container / as an Azure Function.

Three files are worth a read if you adopt or fork it:

- `unified-deployment/main.py` — the orchestrator (window loop, watermark logic, exception handling)
- `unified-deployment/sinks/base.py` — the `AuditSink` interface and the `SinkPartialWriteError` contract
- `unified-deployment/sinks/cosmos_sink.py` — the most-tested sink; HPK + serverless-aware container creation + idempotent upsert

A 30-minute read end-to-end. If something feels wrong for your environment, change it — that's the whole point of publishing it.

Closing

For most Dataverse-to-Azure replication needs, Synapse Link / Fabric Link is the right starting point and you should evaluate them first. The narrower problem this pattern solves is the audit table specifically — where the evidence you usually care about (the `RetrieveAuditDetails` diff) lives outside what a table-level replicator carries, and where the most common business trigger is *"we have years of audit data, we've never deleted any of it, and the storage bill is now a problem."*

If that's where you're sitting — multi-year backlog, Synapse Link / Fabric Link not on the table for whatever reason (often because your analytics platform of record is Snowflake or otherwise outside Azure), looking for a defensible way to move the cold tail to storage you already own *before* you let Dataverse prune anything — this is one option among several. Adopt it, adapt it, or use it as the "here's the shape of the problem" sketch when you talk to your platform team about doing something more substantial.

Comments, forks, and "this is wrong because..." PRs welcome.